2 ноября 2026 года Инженерное дело

Масштабирование Git в OpenAI

Создание системы контроля версий агентного масштаба с инвестициями в экосистему Git.

Аарон Фрил, Тейлор Блау и Тед Найман, сотрудники технического отдела

Загрузка...

Поделиться

За последний год размер нашего монорепозитория и требования к нему увеличились в разы. Стало очевидно, что нам нужно обновить систему контроля версий, чтобы она соответствовала уровню параллельной работы, который обеспечивает Codex.

Мы создали Pando, новый бэкенд для репозиториев Git, не прерывая работу инженеров и не меняя подход нашей команды к использованию Git. Теперь Pando обеспечивает всю разработку в монорепозитории OpenAI. Pando разработан для достижения двух амбициозных целей: во-первых, горизонтально масштабируемая запись, оптимизированная для одновременной отправки данных во множество веток. Во-вторых, максимальная

быстро, используя всю доступную пропускную способность сети.

Мы достигли этих целей, сохранив совместимость с протоколом и клиентом Git, что позволило нашим инженерам и агентам легко интегрировать Pando в существующие рабочие процессы. В производственной среде Pando сократил среднюю задержку отправки почти в 5 раз. Теперь он обрабатывает почти петабайт данных Git в день.

Здесь мы расскажем о причинах, побудивших нас создать новую систему контроля версий, а также о том, почему мы решили сохранить совместимость с Git. Мы сравним нашу систему с распространенными подходами к масштабированию Git и объясним, почему Pando идеально подходит для удовлетворения потребностей OpenAI в масштабируемости. Мы также расскажем о том, что нас ждет дальше, и о том, как мы продолжим вносить свой вклад в развитие Git для поддержки работы с агентами.

Зачем нам нужна была новая система контроля версий

Codex изменил масштабы и темпы разработки в OpenAI, позволив инженерам выполнять гораздо больше задач параллельно. Это означает больше веток и пушей, больше слияний, больше выгрузок и клонов из сред разработки, исследовательских проектов и CI.

Эти требования накладываются друг на друга. Теперь у одного инженера может быть сразу несколько изменений, готовых к непрерывной интеграции, и при каждом пуше может запускаться несколько заданий, обрабатывающих один и тот же коммит.

Примерно с конца прошлого года системы, поддерживающие наш монорепозиторий, начали давать сбои. Мы столкнулись с медленными загрузками и пушами, а также с неудачными слияниями, при этом надежность системы временами падала ниже 90%. Связано ли это с самим Git или с тем, как мы обслуживаем наш монорепозиторий? Git вырос из сообщества, которое обменивалось патчами для ядра по электронной почте. На протяжении более чем двух десятилетий разработчики адаптировали свои инструменты под Git по мере того, как менялся процесс создания программного обеспечения. Когда мы начали сталкиваться с трудностями в нашем

Мы нашли способ сохранить привычный рабочий процесс, перестроив бэкенд таким образом, чтобы он мог параллельно обрабатывать пуш в разные ветки и предоставлять одну и ту же историю многим пользователям одновременно.
Использование Git в качестве основы для нашего рабочего процесса с системой контроля версий означало, что улучшения в клиентском приложении могут быть полезны не только для OpenAI. Мы хотели внести эти улучшения из нашей копии Git в основную ветку, чтобы поделиться ими с более широким сообществом.

Масштабный инжиниринг для Git

Масштабирование системы контроля версий имеет два основных аспекта: множество авторов, отправляющих изменения в разные ветки, и множество читателей, получающих доступ к одной и той же истории. Pando оптимизирован для обоих случаев.

Во-первых, Pando координирует обновления для каждой ветки независимо. Обновления для несвязанных между собой веток могут происходить параллельно, и читатель, загружающий ветку, получает историю, необходимую для этой версии. Компромисс заключается в том, что читатель, просматривающий несколько веток, может видеть их в разное время.

Во-вторых, Pando повторно использует пак-файлы (формат Git для сжатия нескольких объектов) при получении данных, вместо того

некоторые объекты, которые у него уже есть, но сервер может передавать одни и те же байты многим клиентам, практически не затрачивая вычислительных ресурсов.

Эти решения позволяют масштабировать запись в разных ветках и обслуживать запросы на чтение из кэша. Чтобы понять, как они сочетаются друг с другом, давайте рассмотрим, как Git хранит и передает историю изменений.

Руководство по хранению объектов Git

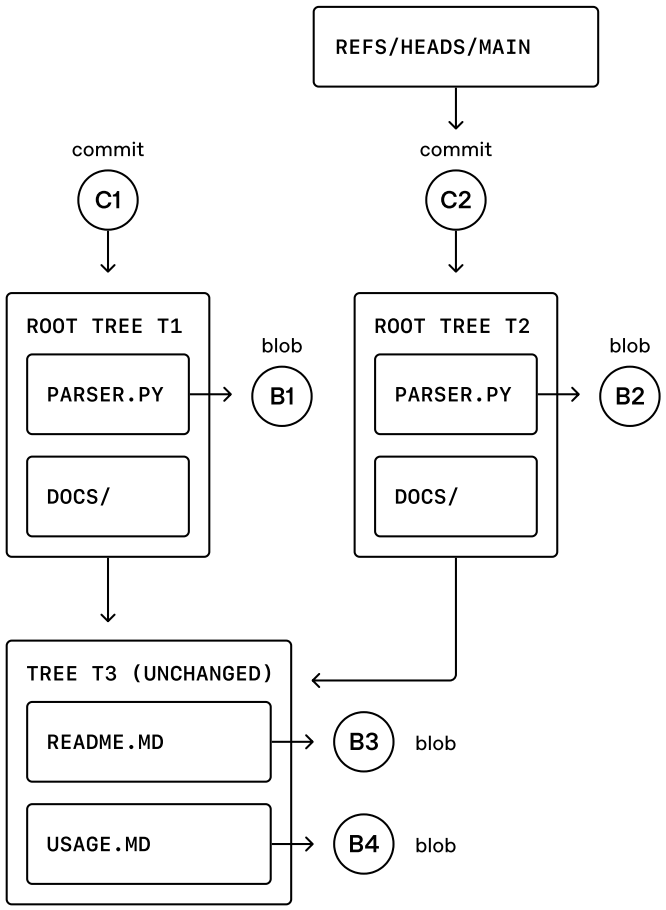
Git должен обеспечивать синхронизацию двух частей репозитория: *объектов* и *ссылок*.

В репозитории Git хранятся снимки файлов и история их соединения в виде графа объектов. Объектом может быть блоб, дерево, коммит или тег. Блобы содержат содержимое отдельного файла, а деревья связывают имена с блобами или другими деревьями, формируя структуру каталогов. Коммиты указывают на корневое дерево и ноль или более родительских коммитов, формируя историю вашего репозитория.

Объекты идентифицируются по хешу их типа и содержимого. Объекты также неизменяемы: при записи измененного файла создается новый блоб, в то время как для неизменных файлов и каталогов можно использовать существующие объекты.

Ветви и теги — это ссылки, или «рефы», указывающие на коммиты. При фиксации в

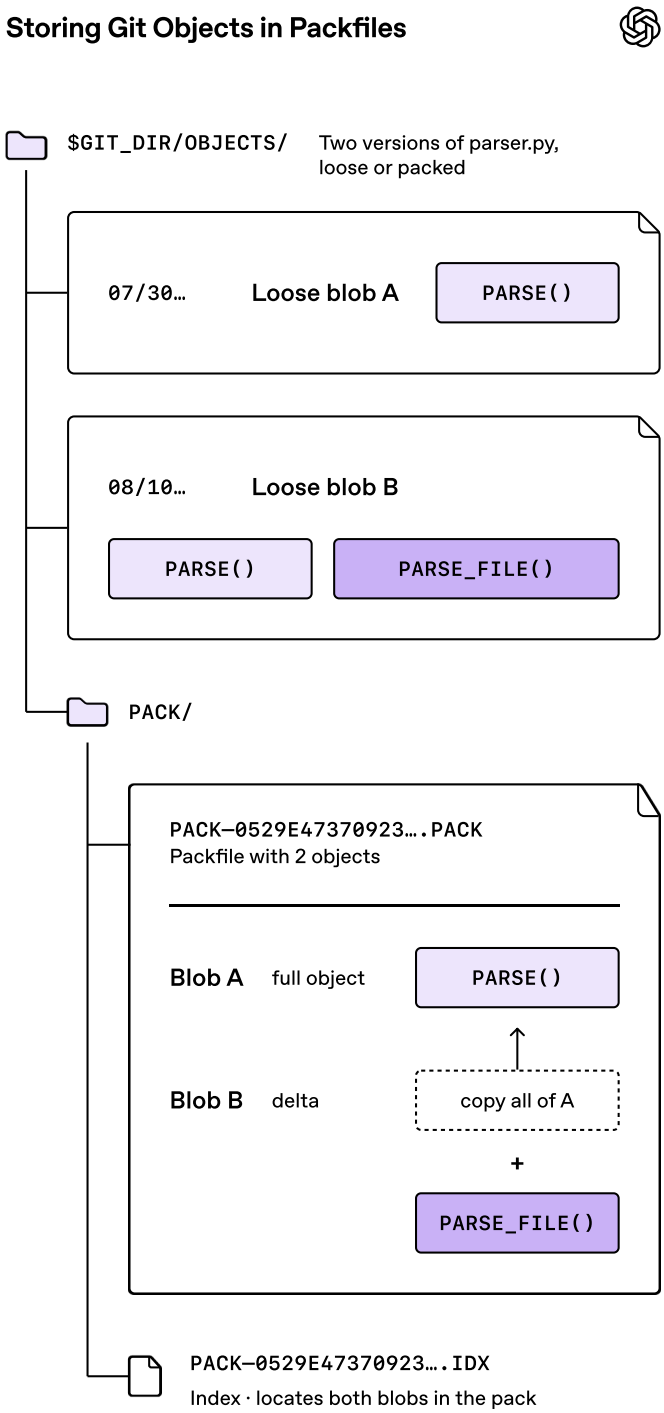
How git objects reference each other



Объекты Git ссылаются друг на друга по идентификатору, коммиты — на деревья, а деревья — на деревья и блобы. Ветвь refs/heads/main указывает на коммит C2, родительским для которого является коммит C1. C1 указывает на корневое дерево T1, где parser.py ссылается на блоб B1. C2 указывает на корневое дерево T2, где parser.py ссылается на блоб B2. В обоих корневых деревьях docs/ указывает на одно и то же дерево T3. Его файл README.md указывает на блоб B3, а файл usage.md — на блоб B4.

Git хранит объекты в «свободном» или «упакованном» виде. Свободные объекты хранятся в отдельных файлах,

вместе в pack-файле, что позволяет хранить похожие объекты в виде дельт друг относительно друга.



LOOSE OBJECTS AND PACKED OBJECTS

Git может хранить блобы в виде отдельных файлов или объединять их, кодируя один блоб как дельту по отношению к другому.

В Git объект *достижим* из коммита, если Git может найти его, следуя по ссылкам от этого коммита через его родительские коммиты, деревья и блобы. Во время выборки клиент сообщает серверу, какие коммиты ему нужны, а какие у него уже есть. Сервер находит объекты, доступные из нужных коммитов, исключает те, которые, как ему известно, уже есть у клиента, и отправляет оставшиеся в виде pack-файла в том же формате, который Git использует на диске. Полное клонирование начинается без каких-либо объектов, поэтому ему нужны все объекты, доступные из клонируемой истории.

Push-уведомление передает объекты в обратном направлении, запрашивая у сервера обновление одной или нескольких ссылок. Клиент использует набор объявленных сервером ссылок, чтобы не отправлять объекты, которые уже есть на сервере. Таким образом, отправляемый пакет может зависеть от существующих объектов: одного пакета недостаточно для восстановления новой истории.

Репликация репозитория Git подразумевает доступность как объектов, так и ссылок на нескольких серверах. Прежде чем реплика сможет предоставить новое значение ссылки, она должна получить доступ к объектам, связанным с этим значением. Сервис также должен координировать конкурирующие *обновления ссылок*, то есть пуши, которые пытаются переместить одну и ту же ветку из одного старого коммита в другой новый. Системы, которые устанавливают единый порядок для всех операций записи в репозиторий, могут привести к тому, что пуши

Существующие шаблоны распределенных сервисов Git

При разработке Pando мы оценивали традиционные подходы к масштабированию системы управления исходным кодом с использованием Git: один из них координирует обновления ссылок между репликами репозитория с помощью трехфазной фиксации (3PC — протокол для координации фиксации на нескольких узлах), а другой упорядочивает обновления с помощью общего журнала упреждающей записи (WAL — запись изменений, сделанных до их применения) в хранилище больших двоичных объектов.

Трехфазная фиксация (3PC)

В смоделированной нами схеме трехфазной фиксации служба Git хранит полную копию репозитория на нескольких репликах. Когда какой-либо сервер получает push-уведомление, входящий pack-файл копируется и параллельно отправляется на все реплики. Чтобы опубликовать push-уведомление, большинство реплик должны принять обновление ссылки. Сервер, управляющий этим процессом, является координатором, и для этого требуется несколько циклов обмена данными между координатором и другими репликами.

реплика блокирует свою копию ссылки и проверяет, что она по-прежнему указывает на коммит, который ожидает клиент. Затем координатор отправляет сообщение о предварительной фиксации репликам, которые его принимают. Как только большинство реплик подтвердят получение сообщения о предварительной фиксации, координатор дает команду на обновление. После этого любая актуальная реплика может объявить о новой конечной точке ветки в своей локальной копии репозитория.

Сохранение полных реплик позволяет выполнять операции чтения локально и гарантирует, что клиенты получат согласованное представление обо всех ссылках, независимо от того, какую реплику они выберут. Таким образом, ссылки обновляются по одной, и для каждого пуша требуется несколько циклов обмена данными, чтобы согласовать обновление ссылок между репликами.

Ожидание кворума снижает производительность реплицированного диска и увеличивает задержку при передаче данных. По мере увеличения объема записи реплики должны успевать обрабатывать входящие пакеты, а также справляться с трафиком чтения.

Журнал упреждающей записи (WAL)

В разработанной нами схеме на основе WAL служба Git использует общий манифест в хранилище больших двоичных объектов для упорядочивания обновлений в репозитории.

больших двоичных объектов и подготавливает обновление ссылки в своей локальной копии репозитория. Чтобы опубликовать push-уведомление, он добавляет в общий манифест, хранящийся в хранилище больших двоичных объектов, запись, в которой фиксируется обновление ссылки и указывается путь к pack-файлу. Затем сервер Git обновляет манифест с помощью операции сравнения и замены. Это приводит к атомарной замене манифеста или отклонению изменения. Если за это время его обновил другой автор, сервер считывает промежуточные записи, проверяет, что обновление ссылки по-прежнему актуально, и повторяет попытку, используя уже загруженный файл пакета.

Авторы могут загружать свои пак-файлы параллельно, но даже при отправке в несвязанные ветки требуется обновление одного и того же манифеста. Время, необходимое для успешного сравнения и замены, ограничивает частоту публикации нового манифеста в репозитории. Хранилище больших двоичных объектов с низкой задержкой позволяет ускорить этот процесс. Пакетная обработка позволяет одновременно обновлять несколько ссылок в одном манифесте, но при этом обновления ставятся в очередь, что увеличивает задержку.

Хранилище BLOB-объектов может получать пакетные файлы параллельно, но обновления общего манифеста по-прежнему конкурируют друг с другом. Увеличение объема хранилища BLOB-объектов само по себе не решает эту проблему. Каждая реплика по-прежнему должна считывать и

их локально. Если входящие записи опережают эту работу, реплики начинают отставать, и клиентам приходится дольше ждать получения новых коммитов.

Проектирование Pando

Это подводит нас к Pando. Pando — это новая архитектура для обслуживания репозиториев Git, предназначенная для одновременной работы с большим количеством клиентов. Вот как она работает:

Переосмысление ограничений

До разработки Pando мы предполагали, что сможем удовлетворить наши требования, не отказываясь от Git, но нам пришлось пойти на компромисс в отношении некоторых вещей, которые большинство систем Git считают обязательными:

- **Согласованность на уровне всего репозитория**: Для получения согласованного снимка состояния всех ссылок на определенный момент времени требуется координация действий во всем репозитории. В системе, которая упорядочивает все обновления ссылок, пуш в несвязанные ветки конкурирует за одну точку координации.
- **Уникальные пак-файлы для каждого клиента**: Если два клиента с разной локальной историей получают одну и ту же ветку, серверу может потребоваться

объекты, которые у них уже есть. Создание таких ответов требует ресурсов процессора и дискового ввода-вывода, что затрудняет масштабирование при большом количестве читателей.

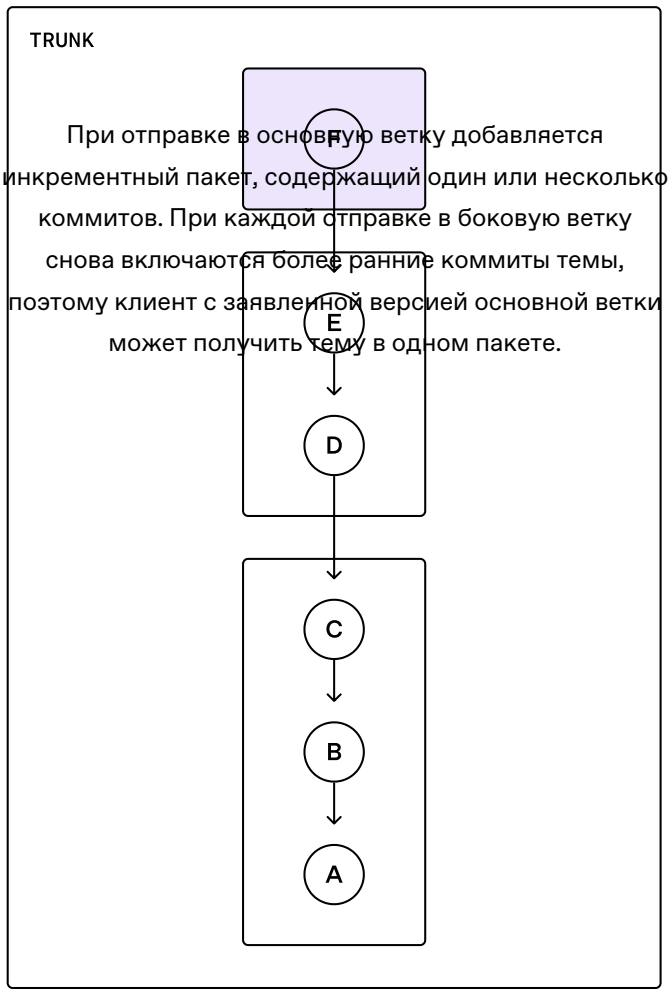
Вместо этого Pando использует модель ослабленной согласованности (причинно-следственной согласованности) для масштабирования операций записи и повторно использует файлы пакетов для масштабирования операций чтения.

Причинно-следственная связь (ПС)

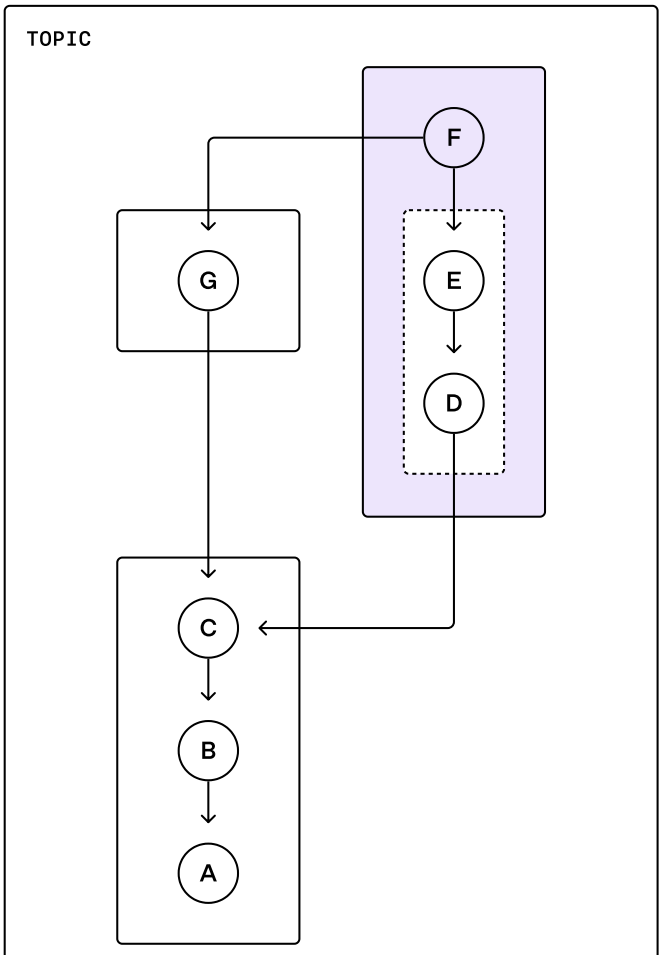
Когда клиент отправляет изменения в ветку, Pando сохраняет входящий пак-файл в виде неизменяемого блоба в хранилище блобов. При каждом перемещении ветки Pando записывает *версию ссылки*, содержащую новую вершину, пакет, из которого она получена, и все предыдущие версии ссылок, от которых зависит пакет, в хранилище метаданных. И хранилище блобов, и хранилище метаданных поддерживают горизонтальную масштабируемость.

Для получения коммита или ссылки из Pando необходимо воспроизвести на клиенте набор версий ссылок (packfiles), чтобы восстановить состояние клиента, создавшего получаемую версию ссылки. Это обеспечивает более слабую форму согласованности во всем репозитории, но при этом гарантирует сериализуемость обновлений отдельных ссылок. Конфликты между клиентами возникают только при одновременной

уверен, что эти данные описывают один и тот же момент времени во всем репозитории.



При отправке в основную ветку добавляется инкрементный пакет, содержащий один или несколько коммитов. При каждой отправке в боковую ветку снова включаются более ранние коммиты темы, поэтому клиент с заявленной версией основной ветки может получить тему в одном пакете.



fetch a reference, the reader observes the history of the writer that produced it. During a push, Pando controls what shared history the client and server agree on, ensuring clients produce more easily shared packfiles.

As a consequence, Pando doesn't provide a point-in-time state of all references during Git operations or an ordering of ref updates within a clone or fetch. Concretely, suppose that branches `foo` and `bar` are both updated during the negotiation of a full clone. During the negotiation, one client may observe `foo` at version 1 and `bar` at version 2, while another client observes `foo` at version 2 and `bar` at version 1. In practice, users and agents are agnostic to the global ordering of ref updates, and instead depend on the order of updates to a single reference.

Comparing Approaches

In this simulation of the three consistency models we configure 1,000 clients to each select a distinct branch and then repeatedly push unique commits, measuring total wall clock duration and push to fetch latency. The latter is an important metric for workloads, as it measures round trip time for users and agents to collaborate or to use other systems which rely on version control. The simulation models hardware ranging from commodity to premium virtual machines and blob storage systems. In this simulation, we've relaxed repository-wide consistency for three-phase commit.

OpenAI

Simulating Git Server Implementations

Follow each writer through repeated pushes, and separately measure when an independent reader can check out each commit.

Infrastructure Fast Playback 30× Writers 1,000 Simulate merge

Batch trunk merges

Three-phase commit — Ready

Simulated time	Mean push to fetch	Writers done
0 s	—	0 / 1,000

0 / 10,000 pushes acknowledged · 0 fetched

Select a square to inspect that writer.

Object-store WAL — Ready

Simulated time	Mean push to fetch	Writers done
0 s	—	0 / 1,000

0 / 10,000 pushes acknowledged · 0 fetched

Select a square to inspect that writer.

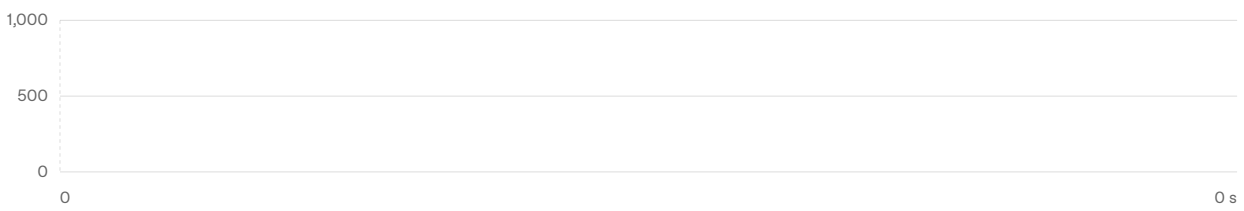
Pando

Simulated time
0 s

0 / 10,000 push

Select a sq

Writer progress through verified fetches Three-phase commit Object-store WAL Pando



OpenAI

Waiting Pushing All pushes acknowledged 0 s Later

Results ↗ Settings

Synthetic teaching models, not production benchmarks. Mean push to fetch covers topic pushes, from the first attempt through retries until an independent reader verifies that commit. It includes waiting for a reader slot and excludes builds and tests. With merges enabled, writer completion includes the required fetches and trunk merge.

Run this simulation comparing Pando with two other approaches. Choose infrastructure from commodity to premium hardware, number of clients, an...

The modeled three-phase commit system shows significantly greater write latency under load as their disks saturate; meanwhile, write-ahead log systems show significantly greater read latency for the same reason, as servers struggle to keep up with reading from the journal. In both real world testing on our monorepo and in this simulation, Pando supports many thousands of clients able to push concurrently without impacting performance under load.

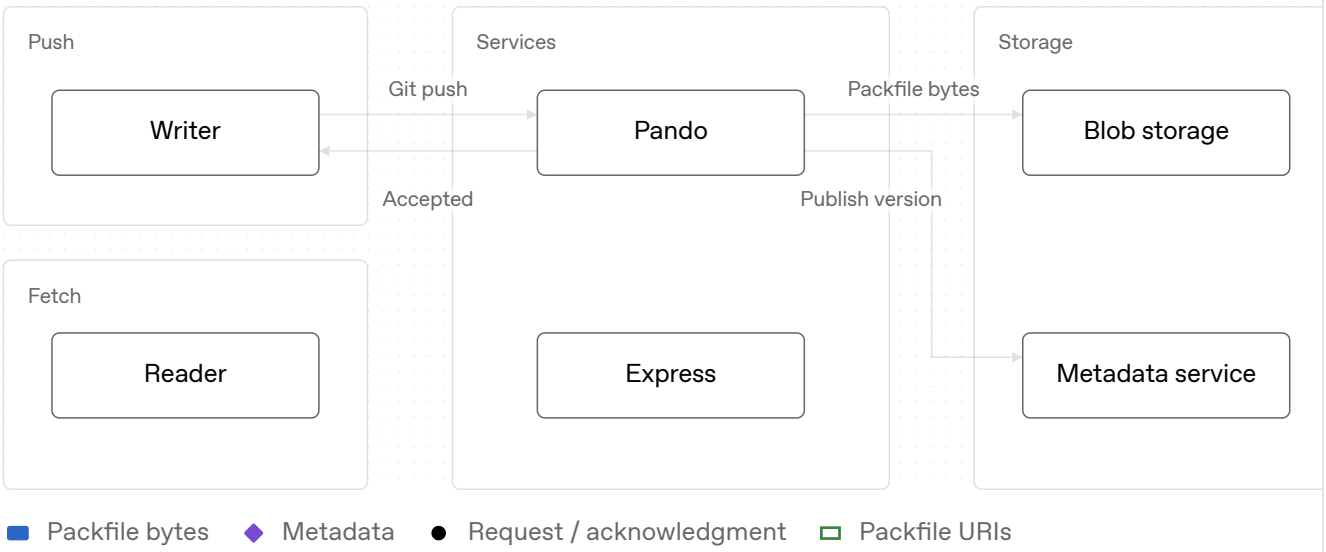
On handling a push, Pando validates the incoming packfile while streaming it to blob storage, without unpacking it into a local repository. When a client begins a fetch Pando selects the stored packs the reader needs, and the reader downloads them. No server needs to apply packfiles to a local repository or perform costly unpack and packing operations. Because these servers primarily perform negotiation and packfile selection, Pando servers can run on commodity hardware.

the network between clients and Pando, and the blob storage service. With enough capacity, clients working on independent branches can each transfer packs as fast as their own connection allows. To serve more clients at that rate, we can add Pando servers and scale the metadata store and blob store alongside them.

Pushing and Fetching a Branch in Pando

Pando stores the writer's packfile in blob storage and records the ref version in metadata service. On fetch, Pando selects the packs the reader needs, and Express serves them.

Stream the push



An agent edits files and creates a commit locally and runs `git push`, which creates a packfile of those changes to update a branch. Pando receives the pack and streams it to blob storage. After validating the incoming push with a single pass decode, Pando uses a multi-key transaction to record a ref version in our metadata store and atomically update the ref pointer. This enables Pando to provide "read-your-own-write" consistency. This entire write

When another client fetches the branch, Pando selects a set of stored packs using the history the client already has. This operation requires $O(\log \Delta)$ metadata reads, Δ being the shortest distance in ref versions between what the client has and what the client wants. The server returns packfile URIs, and the client downloads them through a service, Express, that stores packfiles in regional caches.

This design relies on the three central services of Pando's architecture: pushes, fetches, and compaction.

Scaling pushes

When a client pushes, a conventional Git server is like a poker player showing all of its cards, revealing to the client every branch it has. The client then gathers the commits, trees, and blobs to create a packfile that *excludes* objects it knows the server already has.

Advertising more refs can make the push smaller. It can also make the pack harder to reuse, because another client may be missing objects the writer left out.

Pando takes a different approach. Instead of showing its full hand, it only advertises a select set of branches to the client. This is done for two reasons:

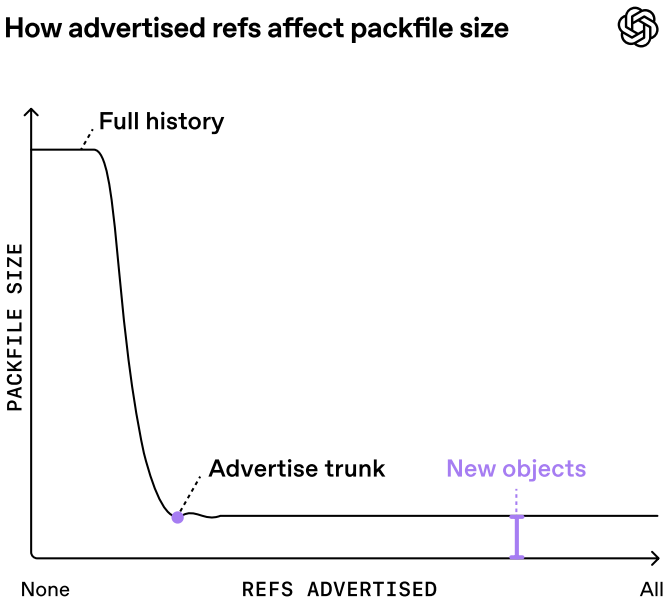
1. Providing a consistent view of every ref is expensive. With millions of branches, the ref advertisement alone can exceed 100 MiB. Providing this requires each server

overhead, or a database query that scales with the number of refs.

2. By providing only a subset of the refs to the client, the client will generate a pack that depends only on what the server explicitly allows the client to exclude. Tracking the dependencies allows Pando to reuse the packfile a client generates in one push by serving it as the response to a subsequent fetch.

As a Git server advertises more refs, clients pushing have more sources from which to exclude objects, enabling them to produce smaller packs. However, reusing these packfiles requires the server to be able to prove the clients fetching have each of the dependencies. Otherwise, the packfile may omit objects the client needs.

On the other hand, if a Git server advertises none at all, the client would generate a packfile containing the repository's full history. That packfile can be used by any other client, but at the significant cost of making fetching any branch effectively a full clone.



Advertising more refs results in smaller packfiles. Most of the size reduction can come from advertising just one.

By default, when a client pushes a new feature branch, Pando advertises only a designated trunk branch as a base. This keeps the advertisement the same size as the repository gains branches, but comes with tradeoffs. When an engineer either pushes to a single branch many times, or stacks one feature branch on top of another the client has to resend everything outside of the trunk.

This causes successive branch pushes to be larger, containing potentially-redundant objects, and uses additional client bandwidth. In our monorepo, the median packfile produced this way was under 20 KiB, and the 99th percentile was under 3 MiB. For us, this was a sensible trade-off because a reader that already has the advertised trunk history can fetch the branch with that one packfile.

client, Pando accepts the push and updates its metadata store. Pando splits the metadata update into synchronous and asynchronous portions. The synchronous metadata update ensures clients can immediately fetch the newly updated commit. This writes a new ref version and updates the ref pointer mapping to that new version storing that new tip, the dependencies used to negotiate the packfile, and a link to the packfile that was pushed.

At the same time, asynchronous work is recorded in the same transaction, such as scheduling compaction. For very large pushes, only a subset of commits are written synchronously. This allows us to require only a constant number of reads and writes to the metadata store, regardless of the size of the push.

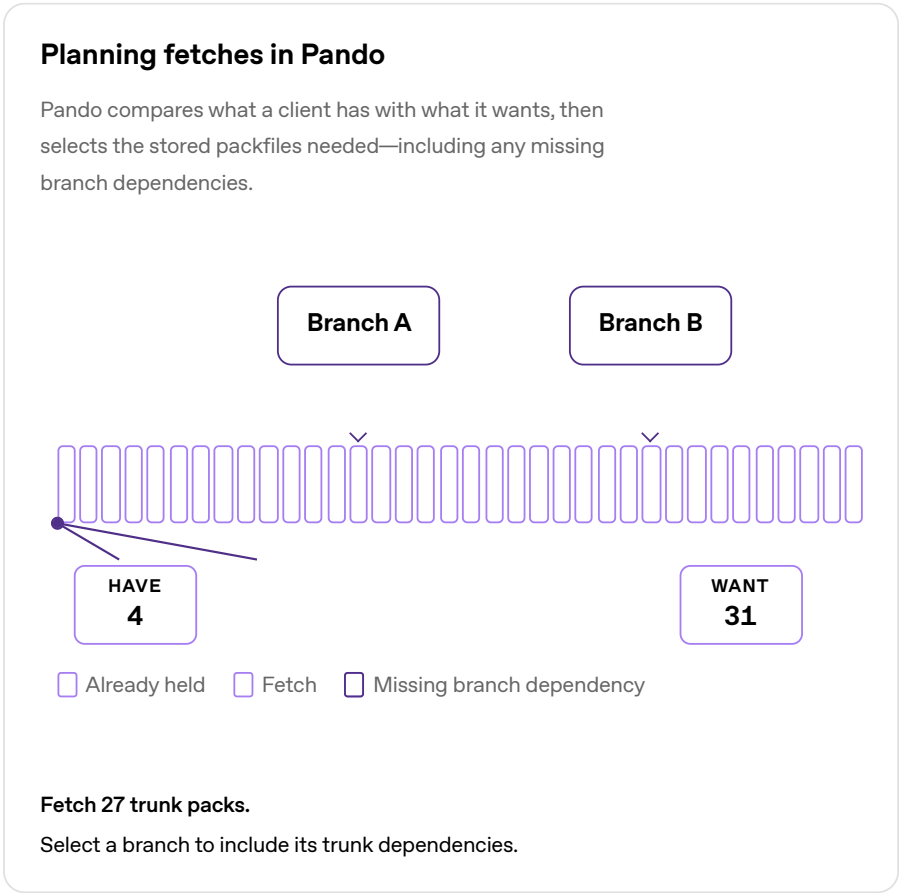
Importantly, Pando servers can accept pushes to unrelated refs independently. The only contended key is the ref pointer, which we guard using an atomic compare-and-swap-like operation. We can scale up write throughput by adding Pando servers, metadata store scaling units, or blob stores.

Line-rate fetch and clone performance

When a client performs a fetch, it provides the server with two sets of commit IDs, known in the protocol as `have`s and `want`s. In a conventional Git server, the server computes the set difference among the objects reachable by the haves and those reachable by the wants, compresses them, and sends the result to the client. While the open-source Git server has a

compute and are generally resistant to caching.

Instead, Pando uses the commit mappings recorded on pushes to look up the client's `have`s and `want`s, resolving them to a set of ref versions that contain those commits. Each of those ref versions links to a packfile previously received by Pando. The server uses this to solve for what the client needs, selecting the packfiles that cover the set of objects the client needs.



Pando maps the Git protocol's haves and wants to a sequence of ref versions. Wanting a side branch implies wanting up to its dependencies on the trunk.

The Git protocol only allows a single packfile to be sent in the HTTP response on a clone or fetch. However, an under-appreciated feature of

separately. Pando uses this exclusively, and transmits an empty packfile inline. To support this feature, OpenAI contributed a number of improvements to `packfile-uri` upstream.

OpenAI systems operate in many geographic regions, so we maintain Pando clusters with local caches in each. A pack delivery service, Express, caches our blob store, and a separate service caches the metadata store. These caches enable Pando to serve requests for a commit ID or a special read of a ref entirely from the cached state in that cluster. We use consistent hashing to route requests for a packfile to Express servers, with cache hit rates of over 99%.

This system of caches provides extremely high throughput, capable of saturating the network limit between clients and Pando. Using these packfile URIs and local caches, we've validated that using only standard cloud provider virtual machines, we see clone and fetch operations saturate 10 Gbps network links.

Compaction and time to first byte

For most Git servers, repacking and garbage collection are necessary to maintain the performance of a growing repository. Both of these steps reclaim space, but are compute- and I/O-intensive.

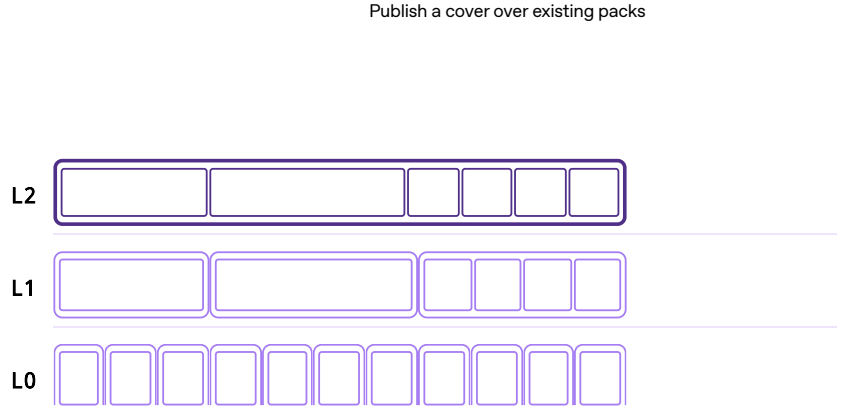
In a fast-moving repository like OpenAI's monorepo, it is infeasible to send an individual packfile URI for every push that has occurred since the last fetch. Each day, a client would

Instead, Pando combines packs in the background to reduce this overhead, adopting a design similar to compaction in log-structured merge-trees. Each push produces what Pando refers to as an “L0” packfile, representing the smallest unit of data Pando can transmit. Over time, background workers collect these packfiles into covers spanning a sequence of ref versions, scaling geometrically from L0. Today, our monorepo uses up to L4 packfile covers.

These covers serve a dual purpose. After a cover is sufficiently full, a compaction worker will claim it and produce a single packfile. This improves the compression ratio by reorganizing the objects inside the packfile. But it also improves fetch negotiation performance by acting as skip list over ref versions. By scanning through indexes at each level simultaneously, Pando can select a set of packs to send to a client in logarithmic time. These packfile covers, repacked or not, enable a more efficient solution to fetch planning, reducing both throughput and latency requirements for Pando.

Compaction in Pando

Small packfiles are grouped into larger covers across levels, improving compression and speeding up fetches.



Compaction combines packfiles from many ref versions in the background, enabling more efficient fetches with fewer files.

Unlike many Git servers, packfiles are kept at every level and not discarded or garbage collected. Packs at every level are used to serve clients continuously, based on the `have`s and `want`s they request in a fetch. And for clients using Pando as an API, reading an arbitrary object from a packfile tends to require reading less data from an L0 than an L4, because it's less likely to be compressed.

Compaction in Pando significantly reduces both the total transfer size sent to clients and enables pack planning to scale sub-linearly in how far behind the client is. This is especially important for large and quickly changing repositories, which we expect to be a new normal in software development.

Building Pando

We built Pando with Codex, moving from kick-off to production, in just 10 weeks. The core of Pando was designed by a single engineer, implemented by teams of agents, and deployed by a handful of engineers. Pando began serving production traffic in April of this year, and is one part of a larger system of source control management tools used at OpenAI.

We employed a wide variety of approaches when validating Pando's design. We applied our

implement Pando in Rust and without legacy dependencies. We wrote a conformance test suite so agents could test their changes against standard Git behavior. We also developed deterministic simulation testing harnesses allowing us to check invariants and properties of the system, enabling us to find and mitigate subtle race conditions and regressions in negotiation and pack planning.

Many models contributed to this system; as new models became available, we used them to improve Pando's performance and reliability. Using GPT-6 Astra, agents wrote Lean and TLA+ to formalize a "theory of Pando" that we used to find and fix bugs in the implementation, and prove properties of the system. Multiple continuous sessions ran for hundreds of hours finding and fixing edge cases in Pando. We also run agents around the clock to red-team Git and Pando using Daybreak-capable models.

Deploying Pando

Our initial deployment of Pando was as a read- and write-through proxy to our existing source control system. This allowed us to keep Pando and the previous system in sync, so we could safely move traffic onto Pando without downtime.

As part of its caching design, Pando can also deduplicate and combine requests before proxying. This allowed us to keep our existing tools healthy as we moved systems to Pando one at a time. As we migrated more workloads to Pando, their pushes populated Pando's cache

In order to optimize performance when using Pando, we began shipping our public distribution of Git to developer machines and within the ChatGPT desktop app. To migrate clients to Pando, we used an internal wrapper around the Git CLI to migrate clients in phases. We created many other tools for handling change requests, code review, and APIs for automations. We migrated hundreds of integrations that depended on legacy APIs, webhooks, and repository downloads alongside Git traffic.

Codex itself helped developers and teams make the move. We created skills for preparing existing checkouts, and for guiding agents to use the new system. As a result, developers could simply ask Codex to migrate their development environments and services.

Switching to Pando improved merge reliability and reduced the latency of the median push by nearly 80%.

Pando can deliver a petabyte of Git data per day with just a handful of CPU cores. Pando has served more than 57 terabytes of Git packs in a single hour, with more than 99% of those bytes from local disk cache.

Reusing packs lets many clients download the same repository history with little computation on the serving nodes. This example shows a single repository, but Pando is designed to support many more. As demand grows, we can scale data delivery and Git negotiation independently.

Contributing back to the Git project

helped us identify bugs and opportunities to improve Git's own performance.

For example, Pando serves reusable packs directly to clients, so downloading and indexing those packs became a central part of making the system fast and reliable. That work surfaced edge cases in concurrent downloads, cleanup after interrupted transfers, and resuming downloads that had already completed. We fixed those paths, added validations when accepting an existing pack for a fetch, and contributed that work back upstream.

We also needed to see where time and bandwidth were going, so we added tracing for `packfile-uri` downloads and byte counts for pack generation, making it easier to tell whether a slow fetch was spending its time generating packs or transferring them.

In OpenAI's Git build, the client can download and index independent packs in parallel. We added the concept of "explicit haves" so a server can advertise a branch's current tip separately from the history a pushing client may leave out of its pack. On Pando, this enables clients to use `--force-with-lease` to check that a feature branch hasn't moved and build its pack against trunk alone. Anyone with that trunk history can then fetch the branch using the same pack.

Several of these improvements are already in upstream Git, and many others in our public repository. We're continuing to work with the Git community to make these improvements useful beyond Pando. You can follow that work in our public patch tracker.

What’s next

By rethinking conventional wisdom on how to scale Git repositories, Pando enables us to scale our monorepo without changing the Git workflows our engineers and agents already use. In our monorepo, Pando has unlocked a greater degree of parallelism, improved merge reliability, and reduced the time it takes to publish code changes. Beyond our monorepo, we’re also exploring other uses for Pando within OpenAI, including workloads where large numbers of agents need to collaborate on a shared Git repository.

As engineers and agents produce more software, more teams beyond OpenAI will encounter the limits we hit in source control. We’re currently exploring ways to share the technology that makes Pando possible with our users.

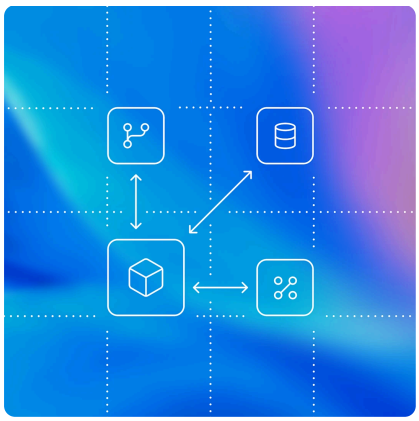
Pando builds on two decades of experience from users and operators of Git repositories. We’ll continue to develop Pando while working with the open-source Git community to share any improvements that benefit Git users more broadly.

2026 Codex

Author

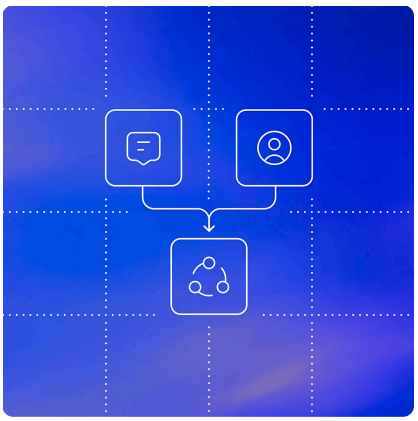
OpenAI

OpenAI



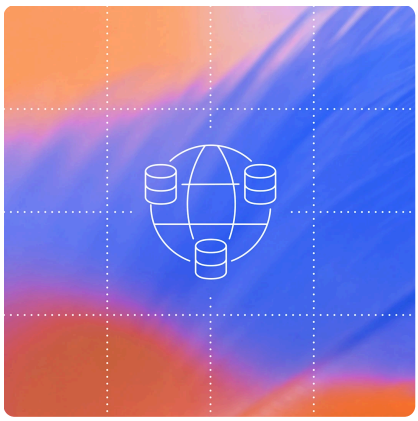
Rapidly scaling online storage to serve over 1 billion ChatGPT users (Part II)

Engineering Oct 23, 2026



How we built ChatGPT's novel ad stack for user trust and relevance

Engineering Oct 21, 2026



Rapidly scaling online storage to serve over 1 billion ChatGPT users

Engineering Sep 11, 2026